

Wait, Spin, or Sleep?

Tail-Latency and CPU Trade-offs in C++20 Queue Waiting

Atharv Bhatt • atharvbhatt10@gmail.com • C++23 implementation / C++20 atomic waiting API

Code + data
atharvv04/Wait-Spin-or-Sleep



Summary

Throughput alone hides the engineering trade-off. I compare three SPSC waiting policies under **open-loop steady and bursty arrivals**, measure p99.9 tail latency and per-thread CPU, and split every event into **producer lateness** and **queue/service latency**.

Three waiting policies

Mutex/CV	mutex-ring Predicate-based condition_variable waits over the same preallocated ring storage.
Spin	Bounded SPSC ring; acquire/release publication; continuously retries while empty or full.
Atomic wait	Same SPSC ring layout with 64-bit change epochs and atomic::wait/notify_one .

Controlled experiment

240 uninstrumented runs	48M measured events
2 standard libraries	5× repetitions / cell

- Clang 18.1.3; **libstdc++ 13** and **libc++ 18 (180100)**.
- 50k, 100k, 250k, 1M events/s; steady + 100-event bursts.
- 200,000 measured events/run; capacity 1,024; fixed work.
- Producer CPU 2, consumer CPU 4; performance governor; boost off.
- Matched ring storage isolates waiting-policy effects.

Measure the right boundary

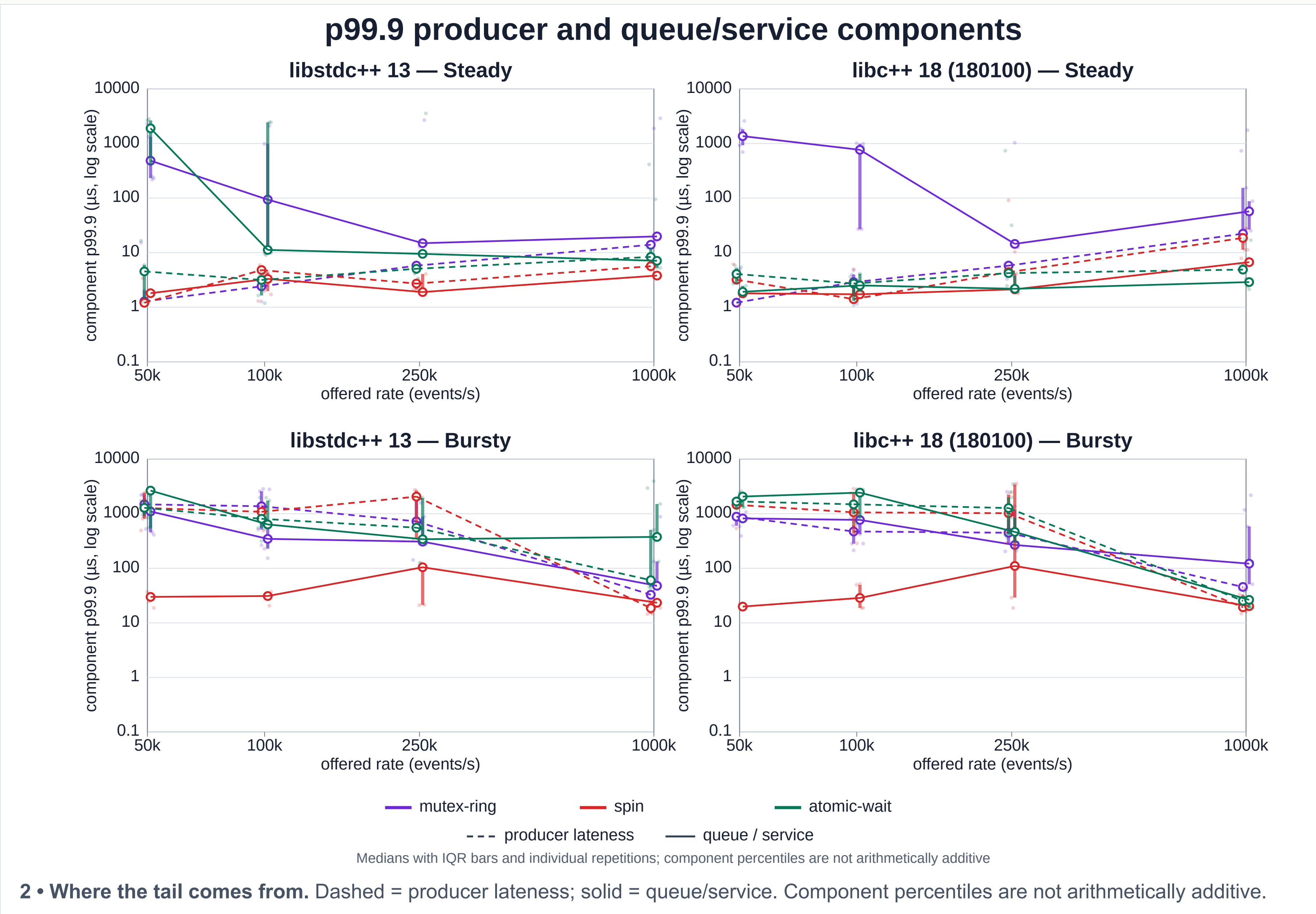
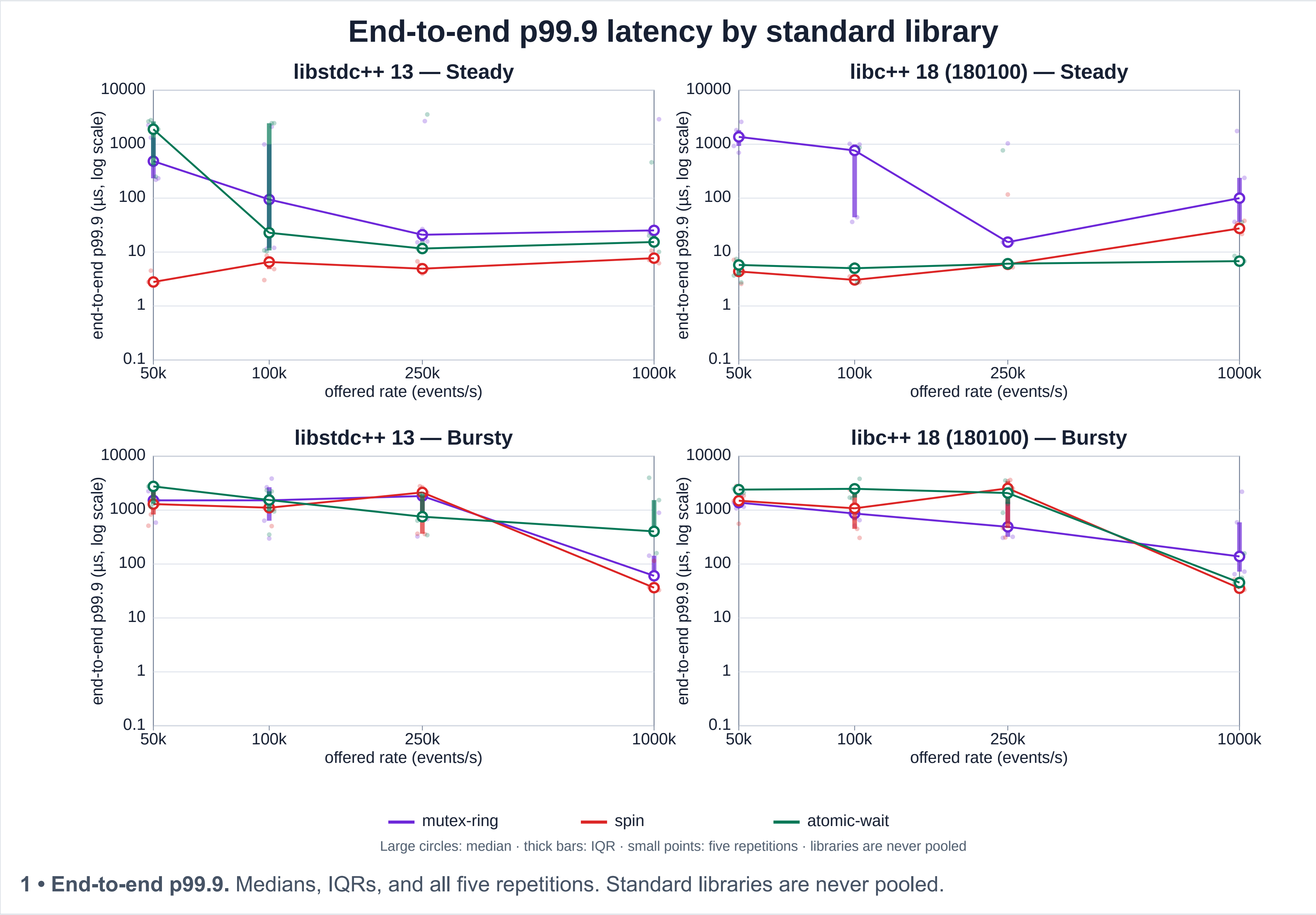
For every event
producer lateness = publish - intended arrival
queue/service = completion - publish
end-to-end = producer lateness + queue/service

Validity: all 48,000,000 current measured events completed with zero loss, duplication, reordering, or latency-decomposition error. A separate 40-run storage control found no universal deque-vs-ring ordering.

One event, three timestamps

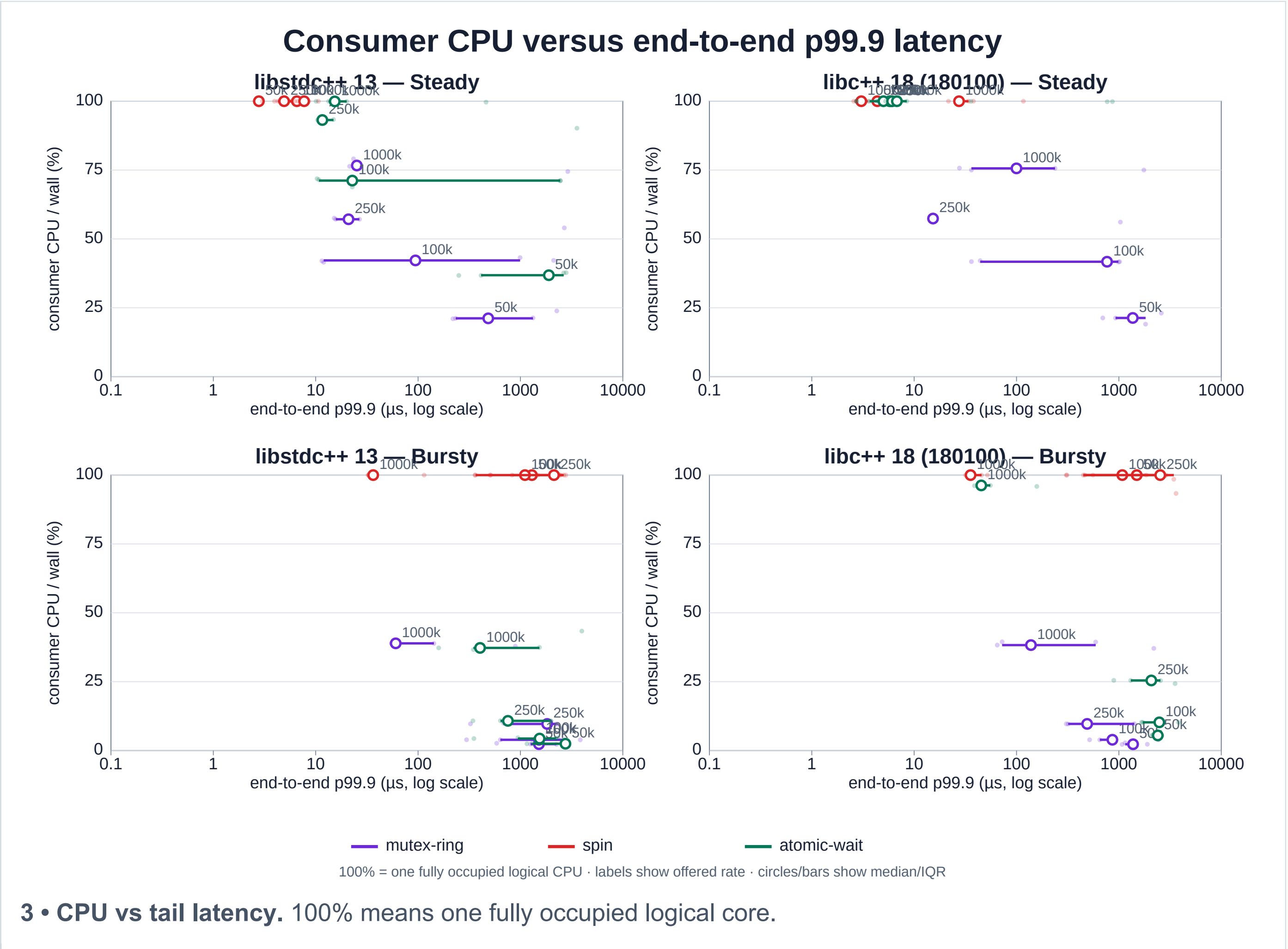


Reproducible artifact. Source, tests, raw CSVs, environment captures, figure generators, methodology, storage-control data, and mechanism traces are public in the QR-linked repository.



The obvious story is incomplete.

- 1 **Same API, radically different result.**
At 50k steady, **atomic-wait** median p99.9 was **1.895 ms** with libstdc++ versus **5.77 μs** with libc++ on this host.
- 2 **Spinning buys latency with a core.**
Spin kept the consumer near **100% CPU** across the current matrix. Low tail latency is not free.
- 3 **Bursty tail latency is not “the queue”.**
Producer scheduling lateness materially contributed to many bursty tails; decomposition prevents misattributing it to synchronization.



Same API ≠ same mechanism
A separate controlled **strace** study counts kernel involvement only; its instrumented latency is not used as performance evidence.

Why can atomic::wait differ so much?

15.4k–18.4k libstdc++ median futex calls / 20k-event run	4–621 libc++ median futex calls / 20k-event run
--	---

Local libstdc++ 13: 64-bit epochs use a 16-entry hashed waiter pool after a short fixed spin.
Local libc++ 18: 256-entry waiter table; polling/yield backoff reaches kernel waiting only after ~64 μs.
Implementation-specific observations, not C++ standard requirements or a universal library ranking.

Practical decision guide

- Spin:** use when microsecond handoff matters and a dedicated core is acceptable.
Mutex/CV: attractive when idle CPU matters; sparse traffic can expose wake-up/scheduler tails.
Atomic wait: benchmark the *actual* standard library you will deploy.

BOTTOM LINE
No universal winner. Measure the workload, the CPU budget, and the standard library you actually ship.